T3000 API & Protocol

Import from Device — Design · built Nov 13, 2026

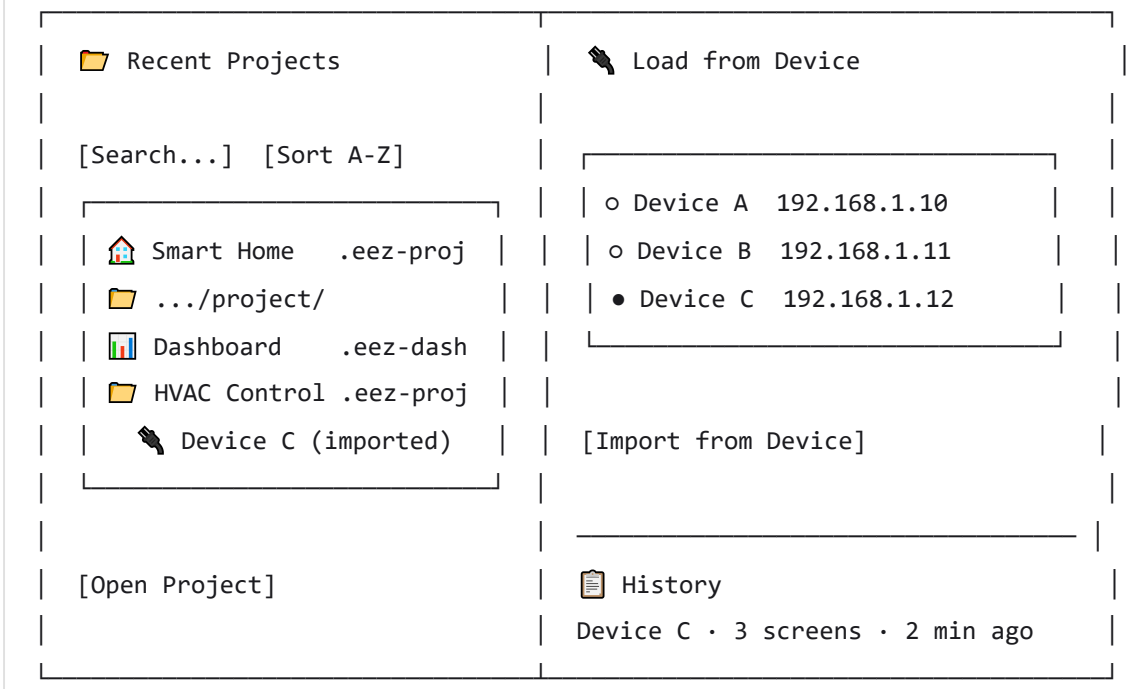
Import from Device — Design

Import firmware screens from a T3000 hardware controller, convert to `.eez-project`, and open in EEZ Studio.

1. UI Layout

Two-column layout on the Open tab:

TEXT



2. Step-by-Step Implementation

Step 1 — List Devices

File: `src/lib/t3-eez-studio/home/open-projects.tsx`

Add a 2-column layout. Right column fetches device list from `/api/devices`:

TYPESCRIPT

```typescript
// New state for right column
const [devices, setDevices] = useState<DeviceInfo[]>([]);
const [selectedDevice, setSelectedDevice] = useState<DeviceInfo | null>(null);
const [importLog, setImportLog] = useState<string[]>([]);
const [importing, setImporting] = useState(false);

// Fetch devices on mount
useEffect(() => {
    fetch('/api/devices')
        .then(r => r.json())
        .then(setDevices)
        .catch(() => setDevices([]));
}, []);
```

Step 2 — Select Device & Click Import

Right column shows radio list of devices. User selects one, clicks [Import from Device]:

TYPESCRIPT

```typescript
<button
    disabled={!selectedDevice || importing}
    onClick={() => startImport(selectedDevice)}
>
    Import from Device
</button>
```

Step 3 — Create Project Skeleton

Create the project folder and `device-import/` staging directory before any network I/O. This ensures the project container exists on disk even if the device never responds.

TYPESCRIPT

```typescript
// Step 3 — Create project skeleton
const projectDir = `project/${device.panel_name}`;
const stagingDir = `${projectDir}/device-import`;
await fetch(`/api/files/mkdir?path=${encodeURIComponent(stagingDir)}`, { method: "POST"
```

Step 4 — Fetch Screens via REST API (Primary) with BACnet Fallback

Uses `DeviceRestClient` (`src/lib/t3-eez-studio/project-editor/build/device-rest-client.ts`) which probes the device for REST API availability and falls back to BACnet through T3000 if unreachable.

Primary path (REST): `GET http://<device-ip>/api/v1/screens` — single request, all screens returned at once. **Fallback path (BACnet):** `POST /api/devices/:id/read-firmware` through T3000 — 200-byte chunks, per-screen.

The screen JSONs are written to `device-import/<name>.json` immediately — if the connection drops mid-transfer, already-fetched screens are safe on disk.

The `device-import/` folder is **kept after import** — it's a local cache. Re-import reads from disk, skipping already-fetched screens (no re-fetch needed).

TYPESCRIPT

```typescript
// Step 4 — Probe device, fetch all screens, save each to device-import/
import { DeviceRestClient } from "project-editor/build/device-rest-client";

const client = new DeviceRestClient();

// Probe: tries REST first, falls back to BACnet if unreachable
const conn = await client.connect(device.ip, device.panel_id, device.serial_number);
addLog(`Connected via ${conn.mode.toUpperCase()}`);

// Fetch all screens in one call (REST) or progressive loop (BACnet)
const result = await client.loadAllScreens();
addLog(`Loaded ${result.screens.length} screens`);

// Save each screen to device-import/ immediately
const stagingScreens: FirmwareScreen[] = [];
for (const screen of result.screens) {
    const screenPath = `${stagingDir}/${screen.name}.json`;
    await fetch(`/api/files/write?path=${encodeURIComponent(screenPath)}`,
        { method: "PUT", body: JSON.stringify(screen.json) });
    stagingScreens.push(screen);
}
```

Step 5 — Build .eez-project from device-import/

Read the raw device JSONs from `device-import/`, convert to `.eez-project` format, save to project root.

TYPESCRIPT

```typescript
// Step 5 — Build .eez-project from staging
const { firmwareToProject } = await import("project-editor/build/firmware-loader");
const project = firmwareToProject(stagingScreens, {
    panel_name: device.panel_name,
    serial_number: device.panel_serial_number,
});

const projectPath = `${projectDir}/${device.panel_name}.eez-project`;
await fetch(`/api/files/write?path=${encodeURIComponent(projectPath)}`,
    { method: "PUT", body: JSON.stringify(project, null, 2) });
```

Step 6 — Open in Editor

TYPESCRIPT

```typescript
// Step 6 — Open project in EEZ editor
settingsController.addItemToMRU(projectPath, { projectType: "LVGL", hasFlowSupport: true
const readResp = await fetch(`/api/files/read?path=${encodeURIComponent(projectPath)}`);
const projectJson = await readResp.json();
await initProjectEditor(tabs, ProjectEditorTab);
```

Step 7 — Transport Layer (Implemented: DeviceRestClient)

The transport is handled entirely by `DeviceRestClient` at `src/lib/t3-eez-studio/project-editor/build/device-rest-client.ts`. It encapsulates both paths:

Path	Load (Import)	Deploy
REST (primary)	`GET http://<ip>/api/v1/screens`	`PUT http://<ip>/api/v1/screens`
BACnet (fallback)	`POST /api/devices/:id/read-firmware` (through T3000)	`POST /api/devices/:id/deploy-firmware` (through T3000)

Connection probe (`client.connect(ip, panelId, serialNumber)`):

1. Tries `HEAD http://<ip>/api/v1/screens` with 2s timeout
2. If reachable → REST mode
3. If unreachable → BACnet fallback (requires panelId + serialNumber)
4. If neither works → error

No new Rust endpoints needed — the REST path goes directly to the ESP32 device. The BACnet fallback path reuses the existing T3000 bridge infrastructure (actions 18/19).

Step 8 — Import History

After import, save a history entry so the user can reopen the drawer:

```typescript
interface ImportHistoryEntry {
    deviceName: string;
    serialNumber: number;
    screenCount: number;
    timestamp: Date;
    log: string[];
}

// Save to localStorage after import
const history = JSON.parse(localStorage.getItem('importHistory') || '[]');
history.unshift({
    deviceName: device.panel_name,
    serialNumber: device.serial_number,
    screenCount: screens.length,
    timestamp: new Date(),
    log: importLog,
});
localStorage.setItem('importHistory', JSON.stringify(history.slice(0, 20)));
```

Step 9 — Badge on Imported Projects

Show `🔌 Device C (imported)` badge in the left column for projects with `importedFrom` metadata:

```typescript
// In open-projects.tsx renderNode
const hasImportMeta = mruItem.filePath && /* check project JSON for importedFrom */;
{hasImportMeta && (
    <span className="import-badge" title={`From ${meta.device} · ${meta.importedAt}`}>
        🔌 {meta.device} (imported)
    </span>
)}
```

---

3. Project Template

Setting	Value
Project type	LVGL with flow support
LVGL version	`9.5.0` (fixed, matches hardware)
File extension	`.eez-project`
Project root	`<data_root>/project/<DeviceName>/`
Project file	`<data_root>/project/<DeviceName>/<DeviceName>.eez-project`
Staging cache	`<data_root>/project/<DeviceName>/device-import/*.json` (kept, never deleted)
Metadata	`importedFrom: { device, serialNumber, importedAt }`

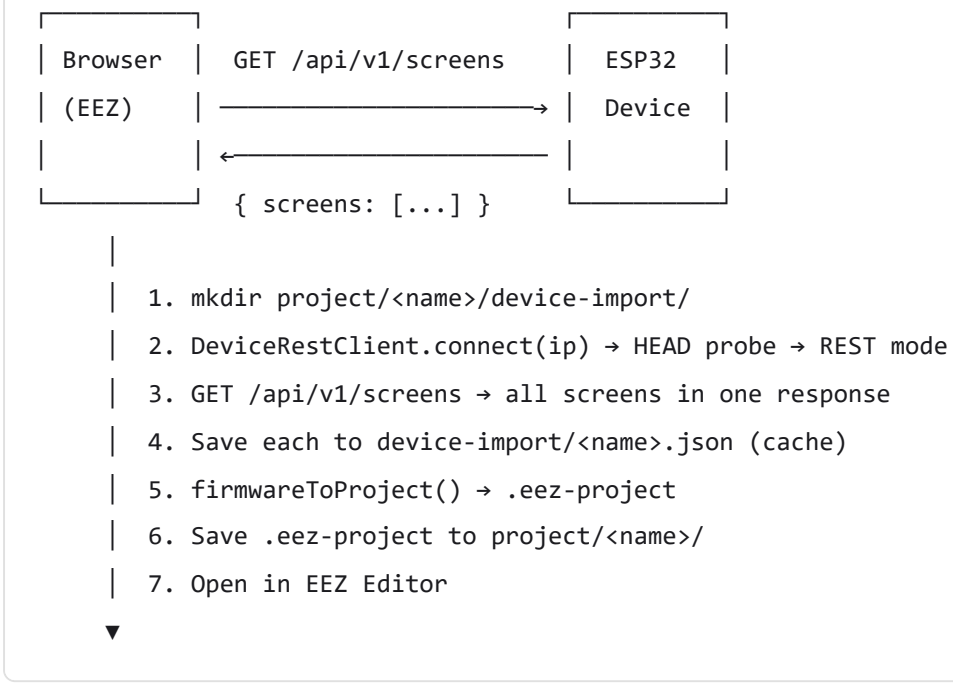
TEXT

```
project/<DeviceName>/
  ├── <DeviceName>.eez-project     ← built from device-import/
  └── device-import/               ← raw device JSONs (cache, kept)
        ├── Screen1.json
        ├── Screen2.json
        └── Screen3.json
```

4. Flow Summary

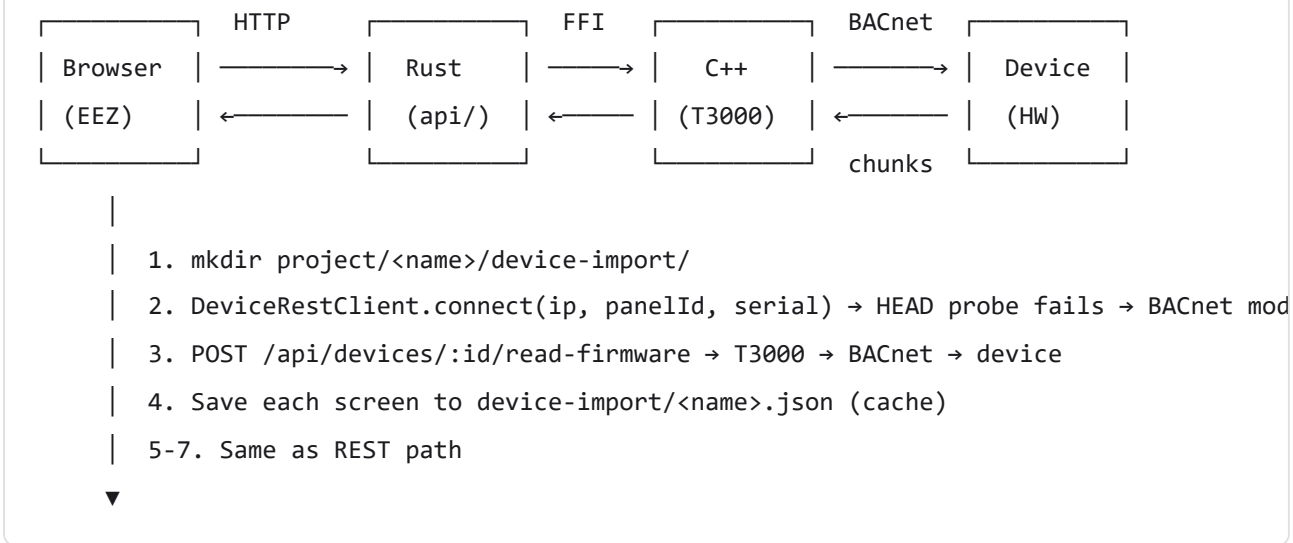
Primary: REST API (direct to ESP32 device)

TEXT



Fallback: BACnet through T3000 (device unreachable via REST)

TEXT



Key behaviors:

- `DeviceRestClient` probes REST first, falls back to BACnet automatically
- `device-import/` folder is created before any network I/O (safe if device offline)
- Each screen saved to `device-import/` immediately on arrival (survives connection drop)

- `device-import/` is never deleted — re-import rebuilds `.eez-project` from cache
- REST path: single `GET` returns all screens at once
- BACnet path: 200-byte chunked transfer through T3000